PHP procédural et Web - Semestre 1

Table of Contents

1. Mode d'emploi . . . 2
   1.1. Convention « cours puis live » . . . 2
   1.2. Démarrer la sandbox . . . 3
   1.3. Convention de typage . . . 4
2. Séance 01 - Git comme carnet de bord . . . 4
   2.1. Démonstration live . . . 4
3. Séance 02 - Premiers programmes, variables et opérateurs . . . 5
   3.1. Un script PHP . . . 5
   3.2. Variables et types . . . 5
   3.3. Exemple live . . . 5
   3.4. Opérateurs essentiels . . . 6
4. Séance 03 - Conditions et logique booléenne . . . 6
   4.1. Comparer . . . 6
   4.2. Classer une mesure . . . 7
   4.3. Démonstration live . . . 7
5. Séance 04 - `foreach` et parcours de tableaux . . . 8
   5.1. Pourquoi une boucle ? . . . 8
   5.2. Tableau indexé . . . 8
   5.3. Tableau associatif . . . 8
   5.4. Démonstration live . . . 9
6. Séance 05 - Fonctions et typage . . . 9
   6.1. Anatomie . . . 9
   6.2. Exemple live . . . 10
   6.3. Bibliothèque . . . 11
7. Séance 06 - HTTP, URL et `$_GET` . . . 11
   7.1. Paramètres d'URL . . . 11
   7.2. Exemple live . . . 12
   7.3. Encoder et échapper . . . 13
8. Séance 07 - Formulaires, `$_POST` et validation . . . 13
   8.1. Du champ à PHP . . . 13
   8.2. Pipeline de traitement . . . 13
   8.3. Démonstration live . . . 15
9. Séance 08 - Tableaux structurés (réactivation) . . . 15
   9.1. Lire une expression imbriquée . . . 16
   9.2. Opérations utiles . . . 17
   9.3. Démonstration live . . . 17

10. Séance 09 - Mini-application procédurale . . . 17
10.1. Responsabilités . . . 18
10.2. Routeur minimal . . . 18
11. Séance 10 - PostgreSQL dans Docker . . . 19
11.1. Démarrer et entrer dans `psql` . . . 19
11.2. Vocabulaire . . . 19
11.3. SQL essentiel . . . 19
12. Séance 11 - Lire PostgreSQL avec PDO . . . 20
12.1. Connexion . . . 20
12.2. Lire . . . 21
13. Séance 12 - Écrire avec PDO . . . 22
13.1. Insertion préparée . . . 22
13.2. Post/Redirect/Get . . . 24
13.3. Transactions . . . 24
14. Séance 13 - Sessions, authentification et hachage . . . 24
14.1. Trois notions . . . 24
14.2. Hacher, pas chiffrer . . . 25
14.3. Exemple live . . . 25
14.4. Session après succès . . . 26
14.5. Protéger et déconnecter . . . 26
14.6. Hors périmètre du semestre . . . 26
15. Fiche réflexe de sécurité . . . 27
16. Fiche réflexe de démonstration . . . 27
16.1. Commandes utiles . . . 27

Ce cours synthétique accompagne les treize séances du module. Chaque notion est illustrée par un exemple court, prévu pour être exécuté en direct dans la sandbox Docker.

1. Mode d’emploi

1.1. Convention « cours puis live »

Pour chaque chapitre :

1. annoncer le problème à résoudre ;
2. faire prévoir le résultat ;
3. ouvrir le fichier indiqué dans `cours/sandbox/` ;
4. exécuter ou recharger ;
5. modifier une seule valeur ;

6. expliquer l'effet observé ;
7. terminer par une erreur volontaire puis sa correction.

1.2. Démarrer la sandbox



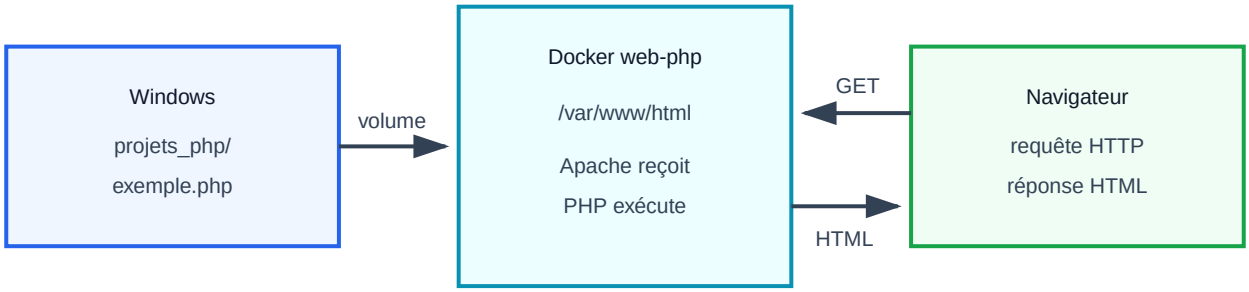
Dans tout ce cours, `<PREPA_BTS>` désigne le dossier racine de l'infrastructure Docker, celui qui contient `docker-compose.yml`. Adaptez-le à votre poste : par exemple `D:\Prepa_BTS` sur la machine de démonstration, ou une autre lettre de lecteur sur le vôtre.

```
cd <PREPA_BTS>
docker compose up -d web-php
docker compose ps web-php
docker compose exec web-php php -v
```

Le dossier Windows `<PREPA_BTS>\projets_php` est monté dans le conteneur sous `/var/www/html`.



Ce module cible **PHP 8.3** (image `php:8.3-apache`) et **PostgreSQL 16**. La commande `php -v` doit afficher une version `8.3.x`. Le cours utilise des fonctionnalités de PHP 8 (types de retour comme `array\|false` ou `?array`, `match`, `password_*`, `filter_var`, `declare(strict_types=1)`), qui exigent au minimum PHP 8.0.



Pour exécuter un exemple en ligne de commande dans le conteneur :

```
docker compose exec web-php php /var/www/html/module-php-s1/cours/sandbox/02-
variables.php
```

Pour les exemples Web :

```
http://localhost:8081/module-php-s1/cours/sandbox/
```



Adaptez `module-php-s1` au nom réel du dépôt étudiant.

1.3. Convention de typage

La séance 2 montre volontairement le typage dynamique de PHP. À partir de la séance 3, les fichiers de démonstration commencent par :

```
<?php

declare(strict_types=1);
```

Les fonctions utilisent des types pour les paramètres et la valeur retournée :

```
function calculerCout(float $prix, int $quantite): float
{
    return $prix * $quantite;
}
```



`strict_types` rend les appels de fonctions plus prévisibles. Il ne transforme pas PHP en langage où toutes les variables doivent être déclarées avec un type.

2. Séance 01 - Git comme carnet de bord

Git enregistre des versions cohérentes du travail.

Commande	Rôle
`git status`	Observer l'espace de travail.
`git diff`	Lire les modifications non indexées.
`git add chemin`	Préparer précisément un changement.
`git commit -m "…"`	Créer une version locale expliquée.
`git push`	Publier les commits locaux.
`git log --oneline`	Lire l'historique condensé.

2.1. Démonstration live

```
git status
git diff
git add cours/sandbox/02-variables.php
git diff --cached
```

Faire distinguer :

- fichier modifié ;

- modification indexée ;
- commit ;
- dépôt distant.



Ne lancez pas `git add .` par réflexe. Vérifiez d'abord les fichiers concernés.

3. Séance 02 - Premiers programmes, variables et opérateurs

3.1. Un script PHP

Un fichier PHP est un fichier texte portant l'extension `.php`. Le code commence par `<?php`.

```
<?php

echo "Bonjour depuis PHP !";
```

Dans un fichier contenant uniquement du PHP, la balise fermante `?>` est omise.

3.2. Variables et types

Une variable :

- commence par `$` ;
- possède un nom explicite ;
- stocke une valeur ;
- peut être observée avec `var_dump()`.

Type	Exemple	Usage
`string`	`"srv-web-01"`	Texte
`int`	`16`	Entier
`float`	`24.90`	Nombre décimal
`bool`	`true`	État vrai/faux

3.3. Exemple live

```
<?php

$nomEquipement = "srv-web-01";
$memoireGo = 16;
```

```php
$coutMensuel = 24.90;
$estActif = true;

$coutAnnuel = $coutMensuel * 12;

echo "Nom : " . $nomEquipement . PHP_EOL;
echo "Mémoire : " . $memoireGo . " Go" . PHP_EOL;
echo "Coût annuel : " . $coutAnnuel . " euros" . PHP_EOL;

var_dump($estActif);
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/02-variables.php`, à coller sur https://3v4l.org ou https://onlinephp.io.

À modifier en direct :

1. passer `$memoireGo` de `16` à `32` ;
2. passer `$estActif` à `false` ;
3. observer `echo` puis `var_dump()` ;
4. retirer un point-virgule, recharger, lire le numéro de ligne, corriger.

3.4. Opérateurs essentiels

Opérateur	Rôle	Exemple
`+`, `-`, `*`, `/`	Calcul	`$prix * $quantite`
`%`	Reste entier	`$nombre % 2`
`.`	Concaténation	`"Nom : " . $nom`
`=`	Affectation	`$actif = true`

4. Séance 03 - Conditions et logique booléenne

Une condition choisit un chemin selon une expression qui vaut `true` ou `false`.

4.1. Comparer

Opérateur	Sens
`===`	Même valeur et même type
`!==`	Valeur ou type différent
`>`, `>=`	Supérieur, supérieur ou égal
`<`, ⇐	Inférieur, inférieur ou égal

Opérateur	Sens
`&&`	ET
`||`	OU
`!`	NON



`=` affecte ; `===` compare.

4.2. Classer une mesure

```
<?php

declare(strict_types=1);

$chargeCpu = 75.0;

if ($chargeCpu >= 90) {
    $etat = "CRITIQUE";
} elseif ($chargeCpu >= 70) {
    $etat = "SURVEILLANCE";
} else {
    $etat = "NORMAL";
}

echo "Charge : " . $chargeCpu . " %" . PHP_EOL;
echo "État : " . $etat . PHP_EOL;
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/03-conditions.php`, à coller sur https://3v4l.org ou https://onlinephp.io.

Trace à faire prévoir :

Charge	État
`95`	`CRITIQUE`
`90`	`CRITIQUE`
`75`	`SURVEILLANCE`
`70`	`SURVEILLANCE`
`69`	`NORMAL`

4.3. Démonstration live

1. tester les cinq frontières ;
2. inverser les tests `>= 90` et `>= 70` ;
3. constater que la première branche vraie arrête la chaîne ;

4. restaurer l'ordre du seuil le plus élevé vers le plus faible.

5. Séance 04 - `foreach` et parcours de tableaux

5.1. Pourquoi une boucle ?

Une boucle répète un traitement.

- `for` : nombre de répétitions connu ;
- `while` : répétition dépendant d'un état ;
- `foreach` : parcours d'un tableau.

Dans ce module, `foreach` est privilégié dès qu'un tableau doit être parcouru.

5.2. Tableau indexé

```
$serveurs = ["srv-web-01", "srv-bdd-01", "srv-cache-01"];

foreach ($serveurs as $serveur) {
    echo $serveur . PHP_EOL;
}
```

5.3. Tableau associatif

```
<?php

declare(strict_types=1);

$chargesCpu = [
    "srv-web-01" => 45.0,
    "srv-bdd-01" => 92.0,
    "srv-cache-01" => 76.0
];

$nombreAlertes = 0;

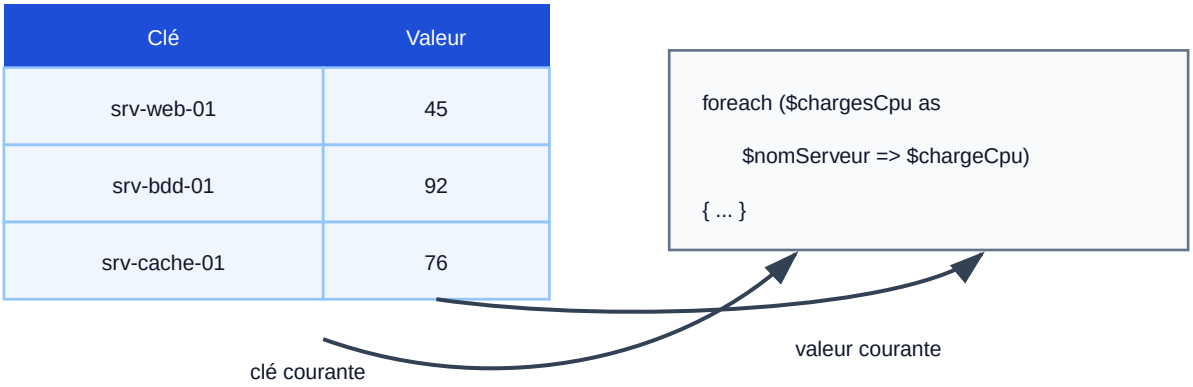
foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    echo $nomServeur . " : " . $chargeCpu . " %" . PHP_EOL;

    if ($chargeCpu >= 70) {
        $nombreAlertes++;
    }
}
```

```
echo "Alertes : " . $nombreAlertes . PHP_EOL;
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/04-foreach.php`, à coller sur https://3v4l.org ou https://onlinephp.io.



À chaque itération :

- `$nomServeur` reçoit la clé ;
- `$chargeCpu` reçoit la valeur ;
- le corps est exécuté ;
- PHP passe automatiquement au couple suivant.

5.4. Démonstration live

1. afficher uniquement les valeurs ;
2. ajouter la clé avec `⇒` ;
3. ajouter un serveur au tableau ;
4. compter les alertes ;
5. remplacer `>= 70` par `>= 90`.

6. Séance 05 - Fonctions et typage

Une fonction nommée regroupe une responsabilité réutilisable.

6.1. Anatomie

```
function determinerEtat(float $chargeCpu): string
{
    if ($chargeCpu >= 90) {
        return "CRITIQUE";
```

```
    }

    if ($chargeCpu >= 70) {
        return "SURVEILLANCE";
    }

    return "NORMAL";
}
```

- paramètre : `$chargeCpu` ;
- type du paramètre : `float` ;
- type retourné : `string` ;
- `return` transmet le résultat à l'appelant.

6.2. Exemple live

```
<?php

declare(strict_types=1);

function determinerEtat(float $chargeCpu): string
{
    if ($chargeCpu >= 90) {
        return "CRITIQUE";
    }

    if ($chargeCpu >= 70) {
        return "SURVEILLANCE";
    }

    return "NORMAL";
}

function calculerMoyenne(array $valeurs): float
{
    if (count($valeurs) === 0) {
        return 0.0;
    }

    return array_sum($valeurs) / count($valeurs);
}

$chargesCpu = [45.0, 92.0, 76.0];

foreach ($chargesCpu as $chargeCpu) {
    echo $chargeCpu . " % : " . determinerEtat($chargeCpu) . PHP_EOL;
}
```

```
echo "Moyenne : " . calculerMoyenne($chargesCpu) . " %" . PHP_EOL;

var_dump(determinerEtat(90.0) === "CRITIQUE");
var_dump(determinerEtat(70.0) === "SURVEILLANCE");
var_dump(determinerEtat(69.0) === "NORMAL");
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/05-fonctions.php`, à coller sur https://3v4l.org ou https://onlinephp.io.

Principes :

- une fonction réalise une tâche ;
- une fonction de calcul retourne, elle n'affiche pas ;
- les données arrivent par les paramètres ;
- les valeurs frontières sont testées.

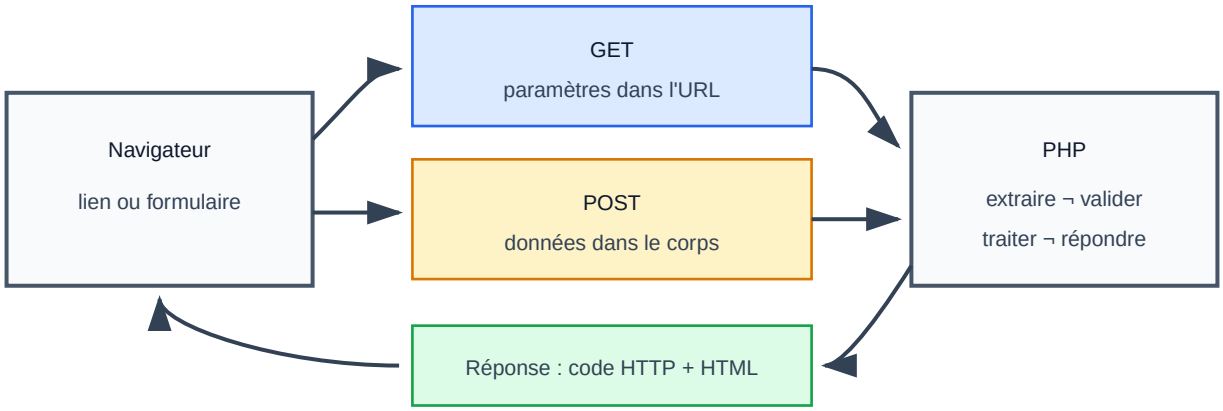
6.3. Bibliothèque

```
require_once __DIR__ . "/lib/supervision.php";
```

`DIR` part du dossier du fichier courant. `require_once` évite une double déclaration.

7. Séance 06 - HTTP, URL et `$_GET`

HTTP fonctionne par requête puis réponse.



7.1. Paramètres d'URL

```
detail.php?nom=srv-web-01&format=court
```

- `?` commence les paramètres ;

- `&` les sépare ;
- `$_GET` est un tableau associatif ;
- l'URL appartient à l'utilisateur : ses valeurs doivent être validées.

7.2. Exemple live

```
<?php

declare(strict_types=1);

$equipements = [
    ["nom" => "srv-web-01", "type" => "serveur"],
    ["nom" => "pc-sio-12", "type" => "poste"],
    ["nom" => "srv-bdd-01", "type" => "serveur"]
];

$typesAutorises = ["tous", "serveur", "poste"];
$typeDemande = $_GET["type"] ?? "tous";

if (!in_array($typeDemande, $typesAutorises, true)) {
    http_response_code(400);
    echo "Type inconnu";
    return;
}

echo "<h1>Équipements</h1>";

foreach ($equipements as $equipement) {
    if (
        $typeDemande === "tous"
        || $equipement["type"] === $typeDemande
    ) {
        echo htmlspecialchars($equipement["nom"]) . "<br>";
    }
}
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/06-get.php`. Faute de requête HTTP en ligne, elle simule `$_GET["type"]` en rejouant plusieurs URL. À coller sur https://3v4l.org ou https://onlinephp.io.

Tester :

```
06-get.php
06-get.php?type=serveur
06-get.php?type=poste
06-get.php?type=inconnu
```

7.3. Encoder et échapper

Fonction	Contexte
`urlencode()`	Valeur placée dans une URL
`htmlspecialchars()`	Texte placé dans du HTML

Une donnée peut nécessiter les deux protections si elle sert à la fois dans un lien et dans son libellé.

8. Séance 07 - Formulaires, `$_POST` et validation

8.1. Du champ à PHP

```
<form action="07-post.php" method="post">
    <input name="nom" required>
    <button type="submit">Envoyer</button>
</form>
```

L'attribut `name="nom"` produit la clé `$_POST["nom"]`.



`required` améliore l'interface, mais peut être contourné. Le serveur valide toujours.

8.2. Pipeline de traitement

1. vérifier la méthode ;
2. extraire avec une valeur par défaut ;
3. normaliser ;
4. valider ;
5. accumuler les erreurs ;
6. traiter seulement si aucune erreur ;
7. échapper lors de l'affichage.

```
<?php

declare(strict_types=1);

$erreurs = [];
$nom = "";
$type = "";
$memoireTexte = "";
$typesAutorises = ["serveur", "poste", "reseau"];
```

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nom = trim($_POST["nom"] ?? "");
    $type = $_POST["type"] ?? "";
    $memoireTexte = $_POST["memoire"] ?? "";

    if ($nom === "") {
        $erreurs[] = "Le nom est obligatoire.";
    }

    if (!in_array($type, $typesAutorises, true)) {
        $erreurs[] = "Le type est invalide.";
    }

    $memoire = filter_var($memoireTexte, FILTER_VALIDATE_INT);

    if ($memoire === false || $memoire < 1 || $memoire > 1024) {
        $erreurs[] = "La mémoire doit être comprise entre 1 et 1024 Go.";
    }

    if (count($erreurs) === 0) {
        echo "<h1>Équipement valide</h1>";
        echo "Nom : " . htmlspecialchars($nom) . "<br>";
        echo "Type : " . htmlspecialchars($type) . "<br>";
        echo "Mémoire : " . $memoire . " Go";
        return;
    }
}
?>
<!doctype html>
<html lang="fr">
<meta charset="utf-8">
<title>Nouvel équipement</title>

<?php foreach ($erreurs as $erreur): ?>
    <p><?= htmlspecialchars($erreur) ?></p>
<?php endforeach; ?>

<form method="post">
    <label>
        Nom :
        <input name="nom" value="<?= htmlspecialchars($nom) ?>">
    </label>

    <label>
        Type :
        <select name="type">
            <option value="serveur">Serveur</option>
            <option value="poste">Poste</option>
            <option value="reseau">Réseau</option>
        </select>
```

```
    </label>

    <label>
        Mémoire :
        <input name="memoire" type="number" value="<?= htmlspecialchars($memoireTexte)
?>">
    </label>

    <button type="submit">Valider</button>
</form>
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/07-post.php`. Elle simule plusieurs soumissions `$_POST` (dont une tentative XSS échappée par `htmlspecialchars()`) et déroule le même pipeline de validation. À coller sur https://3v4l.org ou https://onlinephp.io.

8.3. Démonstration live

Soumettre :

- un nom vide ;
- un type inconnu fabriqué dans les outils du navigateur ;
- une mémoire décimale ;
- `<script>alert(1)</script>` comme nom ;
- une saisie valide.



POST retire les données de l'URL, mais ne les chiffre pas. En production, HTTPS reste indispensable.

9. Séance 08 - Tableaux structurés (réactivation)

Avant d'aborder la base de données, on réactive les tableaux structurés : une ligne de table correspondra à un tableau associatif. Un équipement possède plusieurs propriétés ; un inventaire est donc un tableau de tableaux associatifs.

```
<?php

declare(strict_types=1);

$equipements = [
    [
        "nom" => "srv-web-01",
        "type" => "serveur",
```

```
        "memoireGo" => 16,
        "actif" => true
    ],
    [
        "nom" => "pc-sio-12",
        "type" => "poste",
        "memoireGo" => 8,
        "actif" => true
    ],
    [
        "nom" => "sw-etage-02",
        "type" => "commutateur",
        "memoireGo" => 2,
        "actif" => false
    ]
];

$nombreActifs = 0;
$memoireTotale = 0;
$memoireMaximale = 0;
$nomMemoireMaximale = "";

foreach ($equipements as $equipement) {
    if ($equipement["actif"]) {
        $nombreActifs++;
    }

    $memoireTotale += $equipement["memoireGo"];

    if ($equipement["memoireGo"] > $memoireMaximale) {
        $memoireMaximale = $equipement["memoireGo"];
        $nomMemoireMaximale = $equipement["nom"];
    }
}

echo "Actifs : " . $nombreActifs . PHP_EOL;
echo "Mémoire totale : " . $memoireTotale . " Go" . PHP_EOL;
echo "Maximum : " . $nomMemoireMaximale . " (" . $memoireMaximale . " Go)" . PHP_EOL;
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/08-tableaux.php`, à coller sur https://3v4l.org ou https://onlinephp.io.

9.1. Lire une expression imbriquée

```
$equipements[1]["nom"]
```

Lecture de gauche à droite :

1. `$equipements` : collection ;
2. `[1]` : deuxième élément ;
3. `["nom"]` : propriété `nom`.

9.2. Opérations utiles

Expression	Effet
`$tableau[] = $valeur`	Ajouter à la fin
`$equipement["memoireGo"] = 32`	Modifier
`unset($tableau[2])`	Supprimer
`count($tableau)`	Compter
`array_key_exists("nom", $equipement)`	Tester une clé
`in_array("serveur", $types, true)`	Tester une valeur strictement

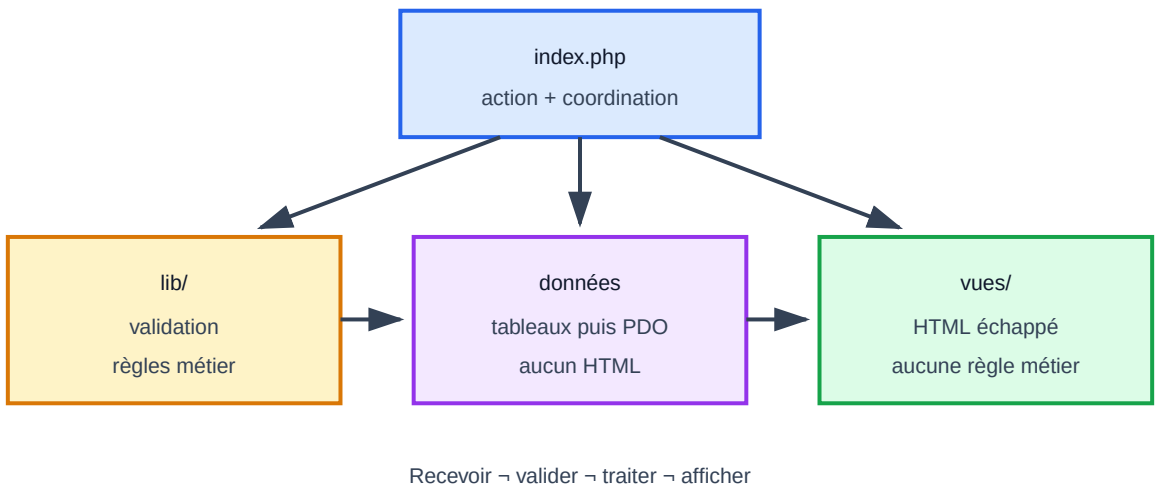
9.3. Démonstration live

Faire trouver :

- le nombre d'équipements actifs ;
- la mémoire totale ;
- l'équipement possédant le maximum ;
- un équipement par son nom ;
- le cas « introuvable ».

10. Séance 09 - Mini-application procédurale

Quand un fichier contient données, validation, règles et HTML, il devient difficile à maintenir.



10.1. Responsabilités

Zone	Responsabilité
`index.php`	Recevoir l'action et coordonner
`lib/`	Fonctions, validation et règles métier
`vues/`	Produire le HTML
`config.php`	Configuration partagée

10.2. Routeur minimal

```
<?php

declare(strict_types=1);

$actionsAutorisees = ["liste", "detail", "nouveau"];
$action = $_GET["action"] ?? "liste";

if (!in_array($action, $actionsAutorisees, true)) {
    http_response_code(404);
    echo "Page inconnue";
    return;
}

if ($action === "liste") {
    echo "Liste des incidents";
} elseif ($action === "detail") {
    $id = filter_var($_GET["id"] ?? null, FILTER_VALIDATE_INT);

    if ($id === false || $id < 1) {
        http_response_code(400);
        echo "Identifiant invalide";
        return;
    }

    echo "Détail de l'incident " . $id;
} else {
    echo "Formulaire de création";
}
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/09-routeur.php`. Elle simule `$_GET["action"]` en rejouant plusieurs requêtes (dont les cas 400 et 404). À coller sur https://3v4l.org ou https://onlinephp.io.

Règles :

- liste blanche des actions ;

- code HTTP `400` pour une entrée invalide ;
- `404` pour une ressource absente ;
- aucune règle métier dans la vue ;
- aucune sortie HTML dans une fonction de calcul.

11. Séance 10 - PostgreSQL dans Docker

Une base rend les données persistantes et interrogeables.

11.1. Démarrer et entrer dans `psql`

```
cd <PREPA_BTS>
docker compose up -d postgres-db
docker compose exec postgres-db psql -U prof_sio -d ecole_db
```

Pour exécuter le fichier de sandbox sans altérer les caractères UTF-8 :

```
docker compose cp .\projets_php\module-php-s1\cours\sandbox\10-sql.sql postgres-
db:/tmp/10-sql.sql
docker compose exec postgres-db psql -U prof_sio -d ecole_db -f /tmp/10-sql.sql
```



Évitez de transmettre un fichier SQL accentué à `psql` avec un pipeline `Get-Content` sous Windows PowerShell 5 : cette chaîne peut modifier l'encodage. `docker compose cp` conserve les octets du fichier.

11.2. Vocabulaire

Terme	Sens
Serveur	Programme PostgreSQL
Base	Ensemble logique de données
Table	Structure en lignes et colonnes
Clé primaire	Identifiant unique d'une ligne
Contrainte	Règle imposée par la base

11.3. SQL essentiel

```
DROP TABLE IF EXISTS incident;

CREATE TABLE incident (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    titre VARCHAR(120) NOT NULL,
    priorite INTEGER NOT NULL CHECK (priorite BETWEEN 1 AND 4),
    clos BOOLEAN NOT NULL DEFAULT FALSE
);

INSERT INTO incident (titre, priorite)
VALUES
    ('Apache arrêté', 1),
    ('Disque presque plein', 2),
    ('Mise à jour planifiée', 4);

SELECT id, titre, priorite
FROM incident
WHERE clos = FALSE
ORDER BY priorite ASC;

UPDATE incident
SET clos = TRUE
WHERE id = 1;
```



Version exécutable en ligne (sans Docker ni PostgreSQL) : `cours/sandbox-online/10-sql.php`, qui rejoue le même scénario en **PDO SQLite en mémoire**. La version PostgreSQL ci-dessus reste la référence. À coller sur https://3v4l.org ou https://onlinephp.io.

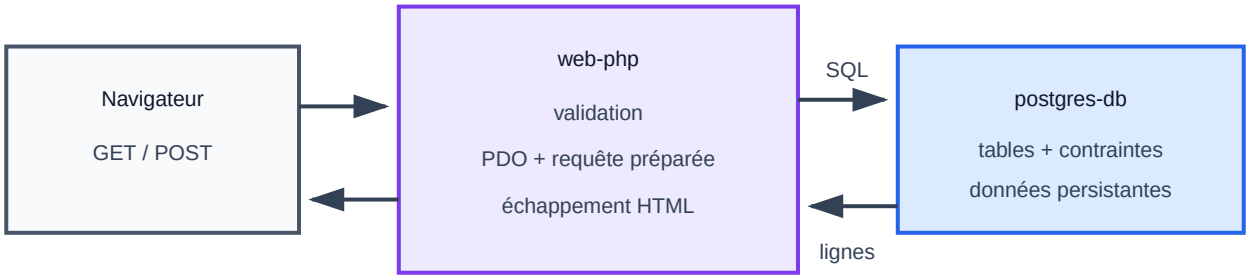
Avant un `UPDATE` ou un `DELETE`, tester exactement le même `WHERE` avec un `SELECT`.



Une mutation sans `WHERE` vise toutes les lignes.

12. Séance 11 - Lire PostgreSQL avec PDO

PDO est l'interface PHP d'accès aux bases. Le pilote utilisé est `pdo_pgsql`.



12.1. Connexion

```
$pdo = new PDO(
    "pgsql:host=postgres-db;port=5432;dbname=ecole_db",
    "prof_sio",
    "SecretPassword123!",
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
    ]
);
```

Dans Docker, `postgres-db` est le nom DNS du service. `localhost` désignerait le conteneur PHP lui-même.

12.2. Lire

```
<?php

declare(strict_types=1);

function obtenirConnexion(): PDO
{
    $hote = getenv("DB_HOST") ?: "postgres-db";
    $base = getenv("DB_NAME") ?: "ecole_db";
    $utilisateur = getenv("DB_USER") ?: "prof_sio";
    $motDePasse = getenv("DB_PASSWORD") ?: "SecretPassword123!";

    return new PDO(
        "pgsql:host=" . $hote . ";port=5432;dbname=" . $base,
        $utilisateur,
        $motDePasse,
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
        ]
    );
}

function obtenirIncidents(PDO $pdo, int $priorite): array
{
    $requete = $pdo->prepare(
        "SELECT id, titre, priorite, clos
         FROM incident
         WHERE priorite = :priorite
         ORDER BY id"
    );
    $requete->execute(["priorite" => $priorite]);

    return $requete->fetchAll();
```

```
}

$priorite = filter_var(
    $_GET["priorite"] ?? 1,
    FILTER_VALIDATE_INT
);

if ($priorite === false || $priorite < 1 || $priorite > 4) {
    http_response_code(400);
    echo "Priorité invalide";
    return;
}

try {
    $incidents = obtenirIncidents(obtenirConnexion(), $priorite);
} catch (PDOException $exception) {
    http_response_code(500);
    echo "Base temporairement indisponible";
    return;
}

foreach ($incidents as $incident) {
    echo htmlspecialchars($incident["titre"]) . "<br>";
}
```



Version exécutable en ligne (sans Docker ni PostgreSQL) : `cours/sandbox-online/11-pdo-select.php`, en **PDO SQLite en mémoire** (schéma et données créés dans le fichier). La logique PDO (`prepare`/`execute`/`fetchAll`) est identique. À coller sur https://3v4l.org ou https://onlinephp.io.

- `query()` : requête fixe sans entrée utilisateur ;
- `prepare()` puis `execute()` : requête contenant des valeurs variables ;
- `fetch()` : une ligne ou `false` ;
- `fetchAll()` : toutes les lignes.



Une donnée venant de la base est échappée avant insertion dans le HTML.

13. Séance 12 - Écrire avec PDO

Une requête préparée sépare le texte SQL des valeurs.

13.1. Insertion préparée

```
<?php

declare(strict_types=1);
```

```
function creerIncident(
    PDO $pdo,
    string $titre,
    int $priorite
): int {
    $requete = $pdo->prepare(
        "INSERT INTO incident (titre, priorite)
         VALUES (:titre, :priorite)
         RETURNING id"
    );
    $requete->execute([
        "titre" => $titre,
        "priorite" => $priorite
    ]);

    return (int) $requete->fetchColumn();
}

$hote = getenv("DB_HOST") ?: "postgres-db";
$base = getenv("DB_NAME") ?: "ecole_db";
$utilisateur = getenv("DB_USER") ?: "prof_sio";
$motDePasse = getenv("DB_PASSWORD") ?: "SecretPassword123!";

$pdo = new PDO(
    "pgsql:host=" . $hote . ";port=5432;dbname=" . $base,
    $utilisateur,
    $motDePasse,
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
);

$pdo->beginTransaction();

try {
    $id = creerIncident(
        $pdo,
        "Incident créé pendant le live",
        2
    );

    echo "Identifiant créé : " . $id . PHP_EOL;
    echo "Démonstration annulée : aucune donnée conservée." . PHP_EOL;

    $pdo->rollBack();
} catch (Throwable $exception) {
    if ($pdo->inTransaction()) {
        $pdo->rollBack();
    }

    throw $exception;
```

```
}
```



Version exécutable en ligne (sans Docker ni PostgreSQL) : `cours/sandbox-online/12-pdo-write.php`, en **PDO SQLite en mémoire**. Noter le portage : PostgreSQL utilise `RETURNING id`, SQLite `lastInsertId()`. À coller sur https://3v4l.org ou https://onlinephp.io.

Ne jamais écrire :

```
$sql = "INSERT INTO incident (titre) VALUES ('" . $titre . "')";
```

13.2. Post/Redirect/Get

Après une écriture POST réussie :

```
header("Location: detail.php?id=" . $id);
exit;
```

Le rafraîchissement répète alors un GET, pas l'insertion.

13.3. Transactions

```
$pdo->beginTransaction();

try {
    // Première écriture
    // Seconde écriture dépendante
    $pdo->commit();
} catch (Throwable $exception) {
    $pdo->rollBack();
    throw $exception;
}
```

Une transaction garantit que l'ensemble réussit ou que l'ensemble est annulé.

14. Séance 13 - Sessions, authentification et hachage

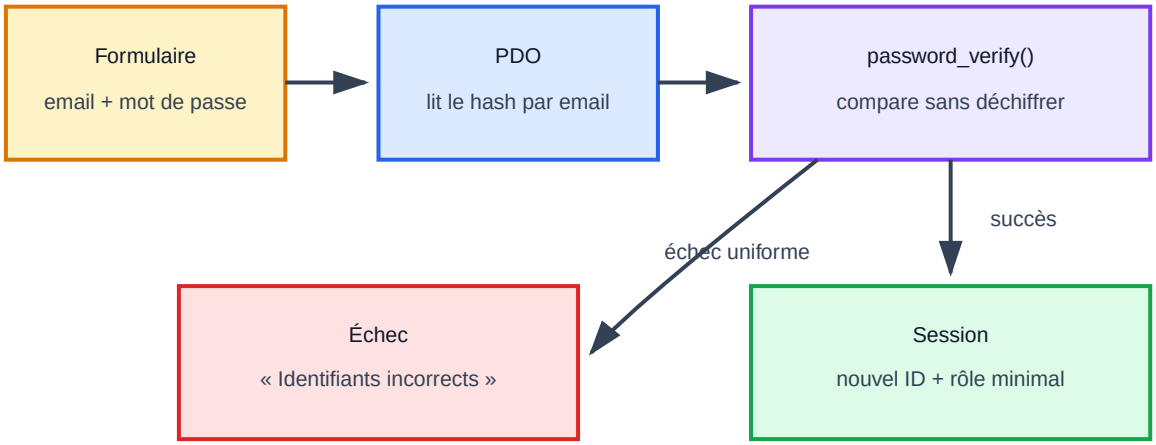
14.1. Trois notions

Identification

annoncer une identité, par exemple une adresse email.

Authentification
: prouver cette identité, par exemple avec un mot de passe.

Autorisation
: vérifier que l'identité authentifiée a le droit d'agir.



14.2. Hacher, pas chiffrer

Un mot de passe doit être vérifié, pas retrouvé.

Mécanisme	But	Réversible
Hachage de mot de passe	Vérifier un secret	Non
Chiffrement	Retrouver une donnée avec une clé	Oui

PHP fournit :

```
$hash = password_hash($motDePasse, PASSWORD_DEFAULT);
$correct = password_verify($motDePasseSaisi, $hash);
```

Ne pas utiliser directement MD5 ou SHA-1 pour les mots de passe. Ne pas fabriquer soi-même le salt.

14.3. Exemple live

```
<?php

declare(strict_types=1);

function authentifier(
    string $motDePasseSaisi,
```

```
    string $hashStocke
): bool {
    return password_verify($motDePasseSaisi, $hashStocke);
}

$motDePasseInitial = "Une-Phrase-Secrete-2026!";
$hash = password_hash(
    $motDePasseInitial,
    PASSWORD_DEFAULT
);

echo "Le mot de passe n'est pas stocké." . PHP_EOL;
echo "Hash : " . $hash . PHP_EOL;

var_dump(authentifier("Une-Phrase-Secrete-2026!", $hash));
var_dump(authentifier("mot-de-passe-incorrect", $hash));
var_dump(password_needs_rehash($hash, PASSWORD_DEFAULT));
```



Version exécutable en ligne (sans Docker) : `cours/sandbox-online/13-auth.php`. Comme `password_hash()` produit un sel aléatoire, la variante en ligne part d'un hash pré-calculé pour garantir une sortie stable. À coller sur https://3v4l.org ou https://onlinephp.io.

14.4. Session après succès

```
session_start();
session_regenerate_id(true);

$_SESSION["utilisateur_id"] = $utilisateur["id"];
$_SESSION["utilisateur_role"] = $utilisateur["role"];
```

Ne stocker ni mot de passe ni hash dans la session.

14.5. Protéger et déconnecter

```
if (!isset($_SESSION["utilisateur_id"])) {
    header("Location: connexion.php");
    exit;
}
```

La déconnexion modifie un état : elle est déclenchée par un formulaire POST.

14.6. Hors périmètre du semestre

Le chiffrement symétrique, asymétrique, hybride, la PKI et JWT sont volontairement différés. Le

pont avec le BLOC 3 porte ici sur les menaces, le hachage spécialisé et le contrôle d'accès.

15. Fiche réflexe de sécurité

Situation	Réflexe
Valeur GET/POST	Valeur par défaut, normalisation, validation
Valeur dans une URL	`urlencode()`
Valeur dans du HTML	`htmlspecialchars()`
Valeur dans SQL	Requête préparée
Mot de passe stocké	`password_hash()`
Mot de passe vérifié	`password_verify()`
Connexion réussie	`session_regenerate_id(true)`
Mutation Web	POST puis redirection
Erreur interne	Message public générique

16. Fiche réflexe de démonstration

Avant chaque live :

- démarrer uniquement les services nécessaires ;
- ouvrir le fichier et le terminal côte à côte ;
- annoncer le résultat attendu ;
- partir d'un cas nominal ;
- tester une frontière ;
- tester une donnée invalide ;
- lire l'erreur plutôt que la masquer ;
- terminer par `git diff`.

16.1. Commandes utiles

```
docker compose ps
docker compose exec web-php php -v
docker compose exec web-php php -l /var/www/html/module-php-s1/cours/sandbox/05-
fonctions.php
git status
git diff
```